EE 446 TinyML – Lab 6 Report

Section 1: Edge Impulse Integration and Deployment

Public Edge Impulse Project: https://studio.edgeimpulse.com/public/998240/latest

1.1 Raw Inputs

Time-series accelerometer data: accX, accY, accZ. Sampling rate: 100 Hz. Window size: 2,000 ms. Stride: 220 ms. Zero-padding enabled.

1.2 NN Classifier Features

Feature type: Spectral features. Total input features: 33.

Top 5 feature:

- accX RMS
- accX Peak 1 Height
- accY Peak 3 Height
- accX Peak 1 Freq
- accY RMS

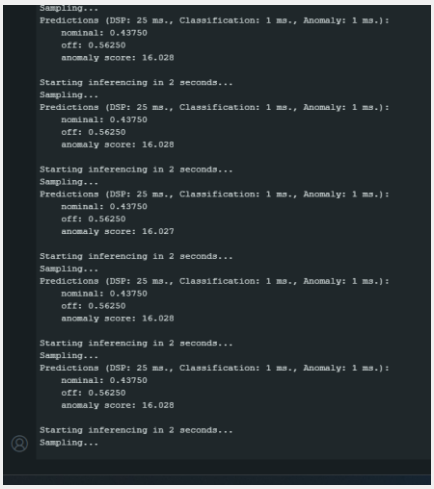
1.3 NN Classifier Outputs

2 output classes: nominal (fan on) and off (fan off). Validation accuracy: 100%, F1: 1.00 for both classes.

1.4 Anomaly Detector Features

Feature type: Spectral features (same as NN classifier). Input axes: accX, accY, accZ. Output: 1 anomaly score (K-means distance from normal clusters).

1.5 Deployment Evidence

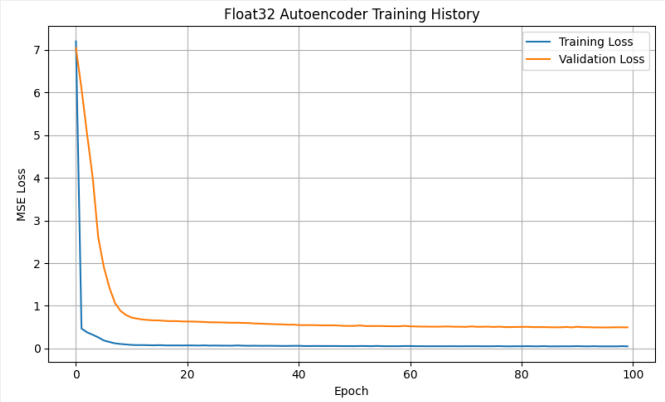


1.6 Interpretation

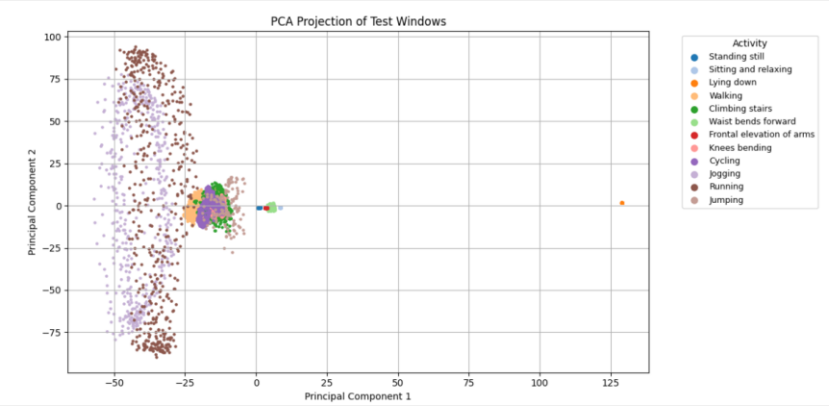
The model ran successfully at ~2s intervals. The high anomaly score (16.028) indicates no fan was attached during inference — readings fell outside trained clusters. The classifier output should not always be trusted: when anomaly score is high, the input is outside the training distribution and classification probabilities are unreliable.

Section 2: Autoencoder Model Results

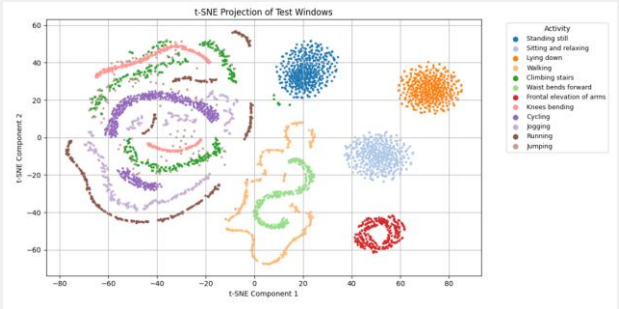
2.1 Training and Validation Loss Curves



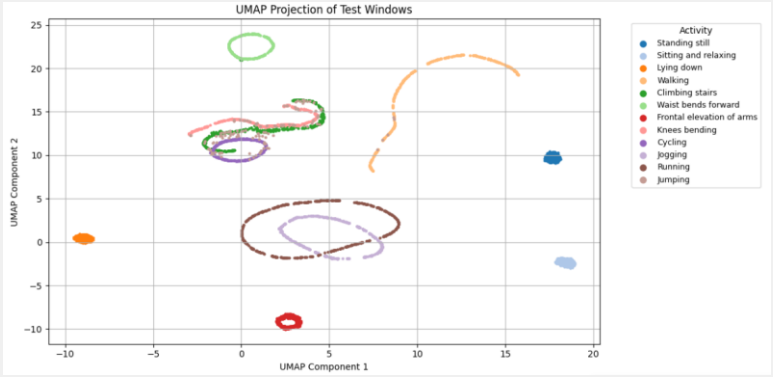
2.2 PCA Projection



2.3 t-SNE Projection



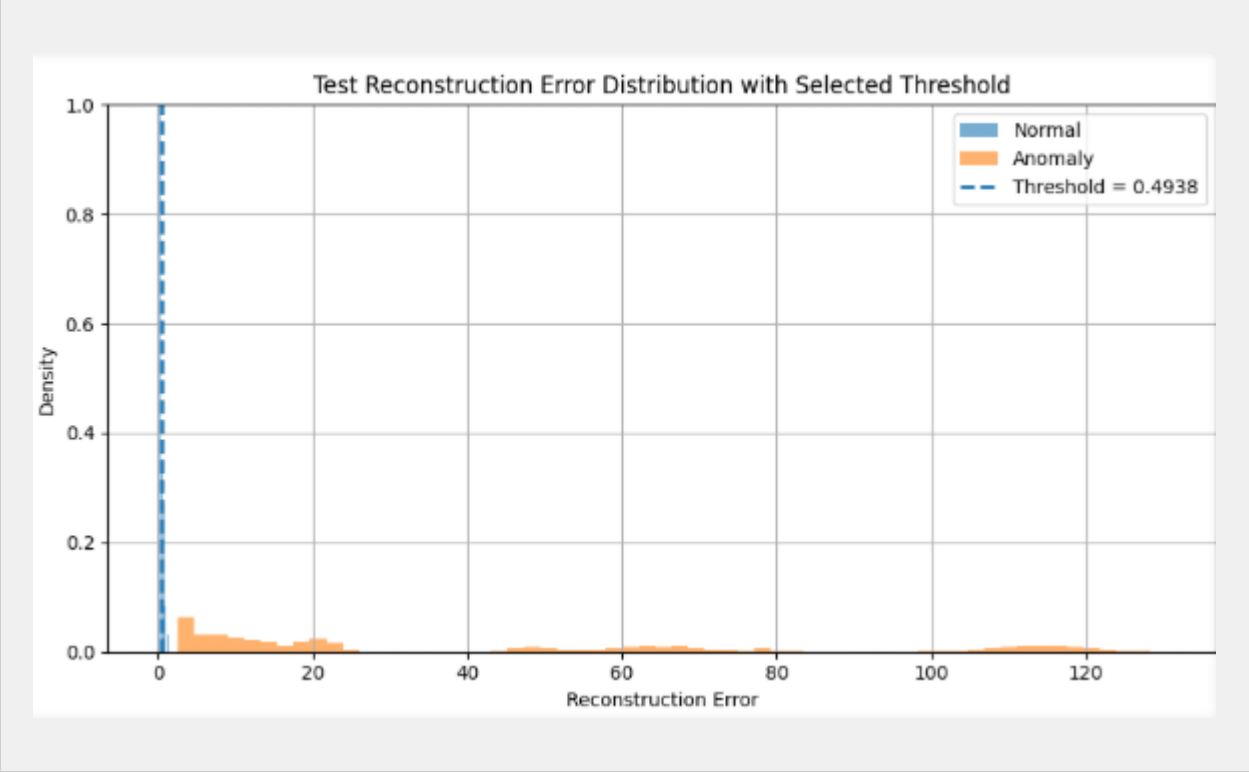
2.4 UMAP Projection



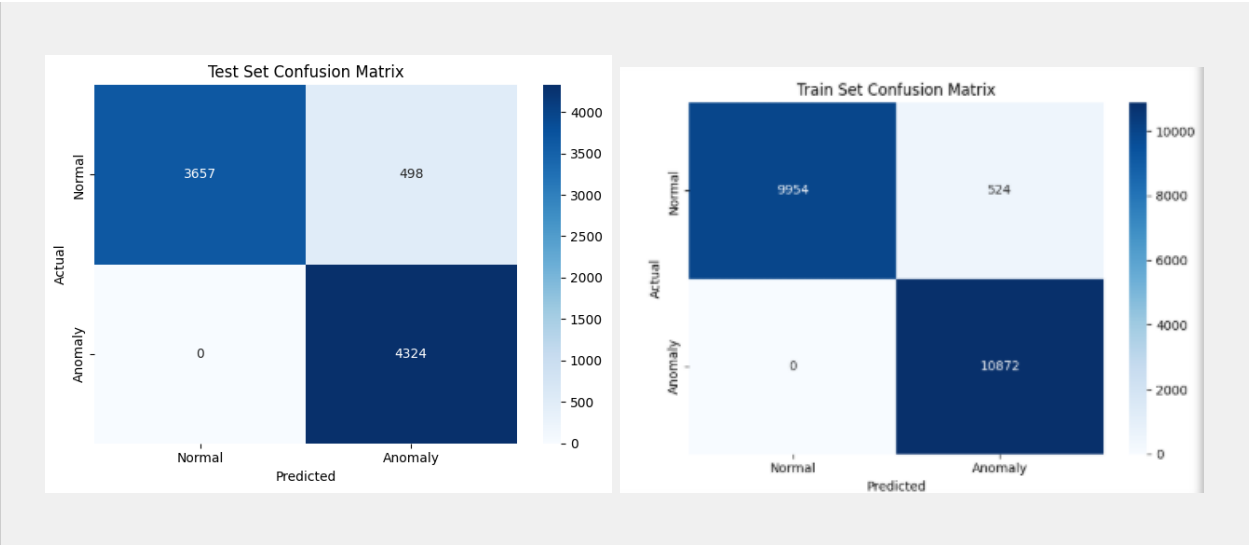
2.5 Reconstruction Error Distribution and Threshold

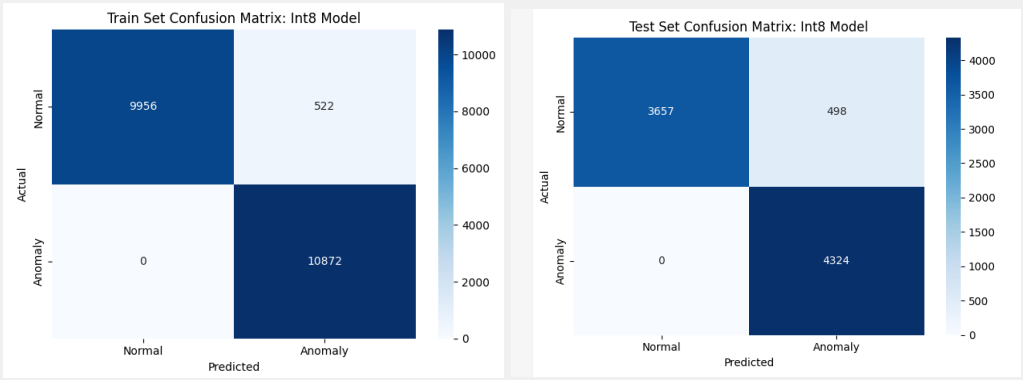
Train normal: mean=0.097, max=1.066 | Train anomaly: mean=42.42, min=2.42

Selected threshold: 0.493785 (95th percentile of normal training errors)



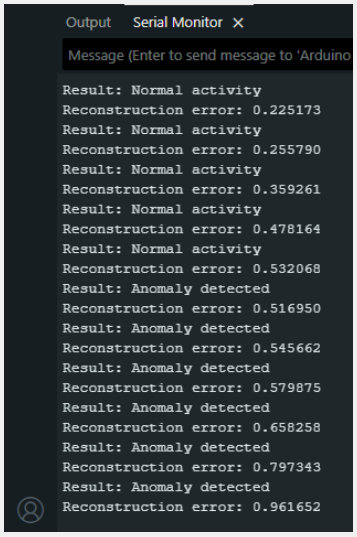
2.6 Confusion Matrix and Classification Report






```
Input quantization: (0.08135765045881271, 18)
Output quantization: (0.07530101388692856, 9)
Train set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      0.95      0.97     10478
 Anomaly (1)       0.95      1.00      0.98     10872

    accuracy                           0.98     21350
   macro avg       0.98      0.98      0.98     21350
weighted avg       0.98      0.98      0.98     21350

Test set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      0.88      0.94      4155
 Anomaly (1)       0.90      1.00      0.95      4324

    accuracy                           0.94      8479
   macro avg       0.95      0.94      0.94      8479
weighted avg       0.95      0.94      0.94      8479
```

2.7 Arduino Serial Monitor Output



Section 3: Discussion Questions

Q1. What threshold did you choose and why?
Threshold of 0.493785, the 95th percentile of normal training reconstruction errors. This ensures 95% of normal windows pass while providing a wide margin below the minimum anomaly error (2.42), giving a clear separation between classes.

Q2. How does reconstruction error distinguish normal vs anomalous activity?
The autoencoder is trained only on normal windows, so it learns to reconstruct them accurately (low MSE). Anomalous windows produce high reconstruction error because the model has never seen those patterns. Thresholding this error gives a binary normal/anomaly decision.

Q3. What notebook information must be preserved for Arduino deployment?
Three things: (1) the int8 model file (autoencoder_model.cc), (2) input quantization parameters (scale and zero point) to convert float32 inputs to int8, and (3) the reconstruction error threshold (0.493785) for the anomaly decision.

Q4. Why must Arduino preprocessing match the notebook?
The model expects exactly 100-sample x 3-axis windows flattened to 300-dimensional vectors. Any difference in window size, axes, or ordering produces inputs the model was never trained on, making reconstruction errors meaningless and anomaly detection unreliable.

Q5. Main limitations on a microcontroller?
Limited to detecting anomalies outside the 6 trained normal classes; brief sub-window anomalies may be missed; int8 quantization introduces rounding that can misclassify borderline windows; fixed memory limits model complexity; no online adaptation possible without retraining